SAMEH_PKG

Oracle PL/SQL HR Management Package

Technical Documentation

Built on the Oracle HR sample schema
13 public routines | 3 private helpers

1. Overview

SAMEH_PKG is a single PL/SQL package that groups a set of HR utilities built on the Oracle **HR** sample schema. It covers salary-increase validation and logging, lookups (address, department, manager), employee counting, date formatting driven by a setup table, top-salary analytics, CSV export/import, and bulk employee creation.

Techniques used across the package: dynamic SQL (`EXECUTE IMMEDIATE` with bind variables), `UTL_FILE`, `BULK COLLECT` / `FORALL`, `LISTAGG` over the data dictionary, OUT parameters, and `RAISE_APPLICATION_ERROR` for error reporting.

Routine index

#	Name	Type	Returns / Purpose
1	increase_emp_sal_check	Function	Y / N - is a proposed raise allowed?
2	increase_emp_sal	Procedure	Applies a valid raise and logs it
3	is_number_fun	Function	Y / N - is the input a valid number?
4	get_full_address	Function	Employee or department address string
5	get_emp_count_fun	Function	Employee count by dept, job, manager, salary category or country
6	is_manager_fun	Function	Y / N - is the employee a manager of employees or departments?
7	get_department_name	Function	Department name for an employee or department id
8	get_manager_full_name	Function	Manager name for an employee or department id
9	my_to_char_fun	Function	Date formatted via setup_table
10	get_top_salaries	Function	Company top salary + OUT: dept, job, location tops
11	export_data_proc	Procedure	Exports any table/view to CSV
12	import_and_update_proc	Procedure	Imports new salaries from CSV and updates employees
13	bulk_insert_emp_proc	Procedure	Bulk-inserts employees from emp_temp

2. Requirements and Setup

Requirements

Oracle Database 11g R2 or higher (LISTAGG), the HR sample schema, and a directory object named **MY_DIR** that the schema user can read and write (needed for UTL_FILE).

Supporting tables

```
-- Log of salary changes (used by increase_emp_sal)
create table sal_log (
    emp_id          number,
    creation_date   date,
    created_by      varchar2(50),
    sal_before      number,
    sal_after       number
);

-- Date formats (used by my_to_char_fun)
create table setup_table (format_id number, data_format varchar2(50));
insert into setup_table values (1, 'DD-MM-YYYY');
insert into setup_table values (2, 'MM-DD-YYYY');
insert into setup_table values (3, 'DD-Mon-YYYY');
commit;

-- Staging table for new salaries (used by import_and_update_proc)
create table emp_new_sals (employee_id number, new_salary number);

-- Staging table for new employees (used by bulk_insert_emp_proc)
create table emp_temp as select * from employees where 1 = 0;
```

Directory object (run as a privileged user)

```
create or replace directory MY_DIR as 'C:\your\path';
grant read, write on directory MY_DIR to hr;
```

Install order

1) Create the four tables above. 2) Create the package specification. 3) Create the package body.

3. Routine Reference

3.1 increase_emp_sal_check

```
function increase_emp_sal_check(emp_id number, increase_flag varchar2,
                                increase_value number) return varchar2;
```

Decides whether a raise is allowed. The new salary must fall between the job's min and max salary (jobs table) and must not exceed the direct manager's salary. An employee with no manager (e.g. the CEO) is only checked against the job range.

Parameter	Description
emp_id	Employee to check
increase_flag	PCT (percentage) or AMT (fixed amount), case-insensitive
increase_value	Percentage or amount to add

Returns: 'Y' if allowed, 'N' if not.

Behaviour / errors: Unknown employee returns the text 'Enter a Valid Employee_id'.

```
select sameh_pkg.increase_emp_sal_check(103, 'PCT', 10) from dual;
```

3.2 increase_emp_sal

```
procedure increase_emp_sal(emp_id number, increase_flag varchar2,
                           increase_value number);
```

Calls increase_emp_sal_check; when the answer is 'Y' it updates the salary, inserts a row into sal_log (employee, date, user, salary before/after) and commits. Otherwise it prints a message through DBMS_OUTPUT and changes nothing.

Parameter	Description
emp_id	Employee to raise
increase_flag	PCT or AMT
increase_value	Value

Behaviour / errors: Invalid employee id prints 'Enter a valid Employee ID'.

```
begin sameh_pkg.increase_emp_sal(103, 'AMT', 500); end;
```

3.3 is_number_fun

```
function is_number_fun(p_number varchar2) return varchar2;
```

Tests whether text can be converted to a number by attempting TO_NUMBER and catching VALUE_ERROR.

Parameter	Description
p_number	Text to test

Returns: 'Y' for 12345, 'N' for 12345x.

```
select sameh_pkg.is_number_fun('12345x') from dual;  -- N
```

3.4 get_full_address

```
function get_full_address(p_id number, p_flag varchar2) return varchar2;
```

Builds 'location_id - street - city - country - region' using dynamic SQL with a bind variable. Outer joins keep employees with no department.

Parameter	Description
p_id	Employee id or department id
p_flag	E = employee, D = department

Returns: Address string.

Behaviour / errors: Unknown id returns 'Enter a valid Employee ID' / 'Enter a valid Department ID'; invalid flag returns 'Enter valid input E or D'.

```
select sameh_pkg.get_full_address(100, 'E') from dual;
```

3.5 get_emp_count_fun

```
function get_emp_count_fun(p_val varchar2, p_ref varchar2) return number;
```

Counts employees by one criterion. A separate lean SQL statement is built per flag so the location joins are used only for countries.

Parameter	Description
p_val	Value to match (varchar2 so any type fits)
p_ref	D dept id | J job id | M manager id | S salary category | C country id

Returns: Employee count (0 when nothing matches).

Behaviour / errors: Invalid flag raises ORA-20001. Salary categories: A <= 5000, B <= 10000, C <= 20000, D above.

```
select sameh_pkg.get_emp_count_fun('IT_PROG', 'J') from dual;
select sameh_pkg.get_emp_count_fun('US', 'C') from dual;
```

3.6 is_manager_fun

```
function is_manager_fun(p_emp_id number, p_flag varchar2) return varchar2;
```

Checks whether the employee id appears as manager_id in employees (E) or departments (D) using a COUNT.

Parameter	Description
p_emp_id	Employee to test
p_flag	E = manages employees, D = manages departments

Returns: 'Y' or 'N'.

Behaviour / errors: Invalid flag ORA-20001; unknown employee ORA-20002.

```
select sameh_pkg.is_manager_fun(100, 'D') from dual;
```

3.7 get_department_name

```
function get_department_name(p_id number, p_flag varchar2) return varchar2;
```

Returns the department name of an employee (E) or of a department id (D). An employee with no department returns 'No Department Assigned'.

Parameter	Description
p_id	Employee id or department id
p_flag	E or D

Returns: Department name.

Behaviour / errors: Invalid flag ORA-20001; unknown id ORA-20002.

```
select sameh_pkg.get_department_name(100, 'E') from dual;
```

3.8 get_manager_full_name

```
function get_manager_full_name(p_id number, p_flag varchar2) return varchar2;
```

Returns the manager's first and last name: the employee's manager (E) or the department manager (D).

Parameter	Description
p_id	Employee id or department id
p_flag	E or D

Returns: Full name.

Behaviour / errors: Invalid flag ORA-20001; unknown id ORA-20002; no manager ORA-20004.

```
select sameh_pkg.get_manager_full_name(166, 'E') from dual;
```

3.9 my_to_char_fun

```
function my_to_char_fun(p_date date, p_format_id number) return varchar2;
```

Formats a date with the pattern stored in setup_table for the given format id.

Parameter	Description
p_date	Date to format
p_format_id	1 DD-MM-YYYY | 2 MM-DD-YYYY | 3 DD-Mon-YYYY

Returns: Formatted date text.

Behaviour / errors: Unknown format id raises ORA-20009.

```
select sameh_pkg.my_to_char_fun(sysdate, 3) from dual;
```

3.10 get_top_salaries

```
function get_top_salaries(emp_id in number, p_out_d out number,
                          p_out_j out number, p_out_l out number) return number;
```

Returns the highest salary in the company and, through OUT parameters, the highest salary in the employee's department, job and location. The first three values come from one query without a join; the location value uses a second query so the join is paid only where needed.

Parameter	Description
emp_id	Reference employee
p_out_d	Top salary in the employee's department
p_out_j	Top salary for the employee's job
p_out_l	Top salary in the employee's location

Returns: Company top salary.

Behaviour / errors: Unknown employee ORA-20002.

```
declare
  d number; j number; l number; top number;
begin
  top := sameh_pkg.get_top_salaries(100, d, j, l);
  dbms_output.put_line(top || ' / ' || d || ' / ' || j || ' / ' || l);
end;
```

3.11 export_data_proc

```
procedure export_data_proc(p_source_name varchar2);
```

Exports any table or view to MY_DIR/<name>.csv. Column names and a query that concatenates every column (NULL shown as 'null') are produced in one LISTAGG query over user_tab_columns; the file starts

with a header line.

Parameter	Description
p_source_name	Table or view name

Behaviour / errors: Bad directory ORA-20001; bad file operation ORA-20002; the file is closed on any error.

```
exec sameh_pkg.export_data_proc('EMPLOYEES');
```

3.12 import_and_update_proc

```
procedure import_and_update_proc;
```

Reads MY_DIR/emp_new_sals.csv (employee_id,new_salary), skips the header, inserts every row into emp_new_sals, then updates employees.salary from that table and commits. Any error rolls back.

Behaviour / errors: Bad directory ORA-20001; bad file operation ORA-20002; other errors ORA-20003.

```
exec sameh_pkg.import_and_update_proc;
```

3.13 bulk_insert_emp_proc

```
procedure bulk_insert_emp_proc;
```

Loads all rows of emp_temp with BULK COLLECT and inserts them into employees with FORALL, then commits. Raises an error if emp_temp is empty.

Behaviour / errors: Rolls back on any failure. Employee id, email and manager id in emp_temp must already be changed so they do not clash with existing keys (see section 5).

```
exec sameh_pkg.bulk_insert_emp_proc;
```

4. Private Helpers and Error Codes

Private helpers (package body only)

Name	Purpose
new_emp_sal(emp_sal, flag, value)	Computes the salary after a PCT or AMT increase. Used by increase_emp_sal_check and increase_emp_sal so the formula lives in one place.
validate_emp_id(p_emp_id)	Raises ORA-20002 when the employee does not exist.
validate_dept_id(p_dept_id)	Raises ORA-20002 when the department does not exist.

Error codes

Code	Meaning
-20001	Invalid flag / empty staging table / invalid directory path (context dependent)
-20002	Invalid employee or department id / invalid file operation
-20003	Unexpected error (message includes SQLERRM)
-20004	Employee or department has no manager
-20009	Invalid format id in my_to_char_fun

Note: -20001 to -20003 are reused with different meanings in different routines. Defining named constants for them in the specification would make callers easier to write.

5. Workflow: Bulk Employee Creation

bulk_insert_emp_proc depends on data prepared in emp_temp. The tested sequence was:

```
-- 1. Copy employees into the staging table (and optionally export to CSV)
insert into emp_temp select * from employees;

-- 2. Change the unique keys so they do not clash
update emp_temp set employee_id = employee_id + 1000;
update emp_temp set email = email || '_NEW';
update emp_temp set manager_id = manager_id + 1000 where manager_id is not null;
commit;

-- 3. Foreign key on manager_id is checked row by row, so a manager may not exist yet.
--    Disable it for the load, then enable it again.
alter table employees disable constraint emp_manager_fk;
exec sameh_pkg.bulk_insert_emp_proc;
alter table employees enable constraint emp_manager_fk;
```

Errors seen during testing: ORA-02291 (EMP_MANAGER_FK, parent key not found) when a manager row is inserted after its employee, and ORA-00001 (EMP_EMAIL_UK) when emails were not changed first.

6. Design Notes

Topic	Decision
Flag handling	The flag is upper-cased once into a local variable and validated before any query runs.
Dynamic SQL	Always with bind variables (USING) so statements can be shared and are not open to injection through values. Only table names (export_data_proc) are concatenated.
Validation	Shared in validate_emp_id / validate_dept_id instead of being repeated per function.
Performance	get_top_salaries uses two queries rather than one joined query so the join is not paid for the three values that do not need it; bulk_insert_emp_proc uses BULK COLLECT + FORALL.
COUNT behaviour	COUNT never raises NO_DATA_FOUND, so an unknown value returns 0 rather than an error.

Topic	Decision
export_data_proc	p_source_name is concatenated into SQL; callers should pass trusted table names only (DBMS_ASSERT.SIMPLE_SQL_NAME would harden this).

Known improvements still open

Item	Detail
import_and_update_proc	Declares two collection types but inserts row by row; collect values and use FORALL. Truncate emp_new_sals first so a second run does not duplicate rows.
new_emp_sal	Returns NULL for a flag other than PCT / AMT; add an ELSE that raises an error.
increase_emp_sal_check	Passes the original flag to new_emp_sal instead of the upper-cased variable (harmless, because new_emp_sal upper-cases again).
get_manager_full_name	The final ELSE after the flag check is unreachable and can be removed.